COMPONENTE CURRICULAR:	Programação de Sistemas II
NOME COMPLETO DO ALUNO:	Murillo Ressineti Silva
RA:	10732430

1. Classe Abstrata Pessoa

Como definido no material, classes abstratas servem como um modelo para outras classes e não podem ser instanciadas diretamente.

```java
abstract class Pessoa {
    private String nome;
    private String cpf;
    private String email;

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }

    // Métodos Getters e Setters (Encapsulamento) [cite: 581, 582]
    public String getNome() { return nome; }
    public String getCpf() { return cpf; }
    public String getEmail() { return email; }

    // Método abstrato que deve ser implementado pelas subclasses [cite: 660]
    public abstract void exibirInformacoes();
}
```

2. Subclasse Aluno

A classe Aluno utiliza a palavra-chave extends para herdar de Pessoa.

```java
class Aluno extends Pessoa {
    private String ra;
    private String curso;
    private int anoIngresso;

    public Aluno(String nome, String cpf, String email, String ra, String curso, int anoIngresso) {
        super(nome, cpf, email); // Chama o construtor da superclasse [cite: 614]
        this.ra = ra;
        this.curso = curso;
        this.anoIngresso = anoIngresso;
    }

    @Override
    public void exibirInformacoes() {
        System.out.println("=== Dados do Aluno ===");
        System.out.println("Nome: " + getNome() + " | RA: " + ra + " | Curso: " + curso);
    }
}
```

3. Interface Trabalhador e Classe Professor

A interface funciona como um "contrato" que obriga a implementação de certos métodos.

```java
interface Trabalhador {
    void trabalhar();
}

class Professor extends Pessoa implements Trabalhador {
    private String registroProfissional;
    private String titulacao;
    private int limiteHoras;

    public Professor(String nome, String cpf, String email, String registro, String titulacao, int limiteHoras) {
        super(nome, cpf, email);
        this.registroProfissional = registro;
        this.titulacao = titulacao;
        this.limiteHoras = limiteHoras;
    }

    @Override
    public void trabalhar() {
        System.out.println("O professor está ministrando aulas.");
    }

    @Override
    public void exibirInformacoes() {
        System.out.println("=== Dados do Professor ===");
        System.out.println("Nome: " + getNome() + " | Registro: " + registroProfissional + " | Titulação: " +
        titulacao);
    }
}
```

4. Classe Coordenador

Esta classe demonstra a herança múltipla de comportamentos, pois herda de Professor e implementa a interface Trabalhador.

```java
class Coordenador extends Professor {
    private String departamento;

    public Coordenador(String nome, String cpf, String email, String registro, String titulacao, int limiteHoras,
    String departamento) {
        super(nome, cpf, email, registro, titulacao, limiteHoras);
        this.departamento = departamento;
    }

    @Override
    public void trabalhar() {
        System.out.println("O coordenador está gerenciando o departamento de " + departamento + ".");
    }

    @Override
    public void exibirInformacoes() {
        super.exibirInformacoes();
        System.out.println("Cargo: Coordenador do Departamento de " + departamento);
    }
}
```

5. Classe de Teste (Main)

A classe Aluno utiliza a palavra-chave extends para herdar de Pessoa.

```java
public class SistemaAcademico {
    Run | Debug
    public static void main(String[] args) {
        Aluno aluno = new Aluno("João Silva", "123.456.789-00", "joao@mackenzie.br", "22.12345-6", "ADS", 2022);
        Professor professor = new Professor("Thiago Traue", "987.654.321-11", "thiago@mackenzie.br", "PR1234",
        "Mestre", 40);
        Coordenador coord = new Coordenador("Ana Souza", "111.222.333-44", "ana.coord@mackenzie.br", "PR5555",
        "Doutora", 44, "Tecnologia");

        // Testando polimorfismo e exibição [cite: 623, 641]
        aluno.exibirInformacoes();
        System.out.println();

        professor.exibirInformacoes();
        professor.trabalhar();
        System.out.println();

        coord.exibirInformacoes();
        coord.trabalhar();
    }
}
```